

Федеральное государственное автономное
образовательное учреждение высшего образования
Национальный Исследовательский Университет ИТМО
Факультет Программной Инженерии и Компьютерной Техники

**Курсовая работа
по Информационным системам
Часть 2**

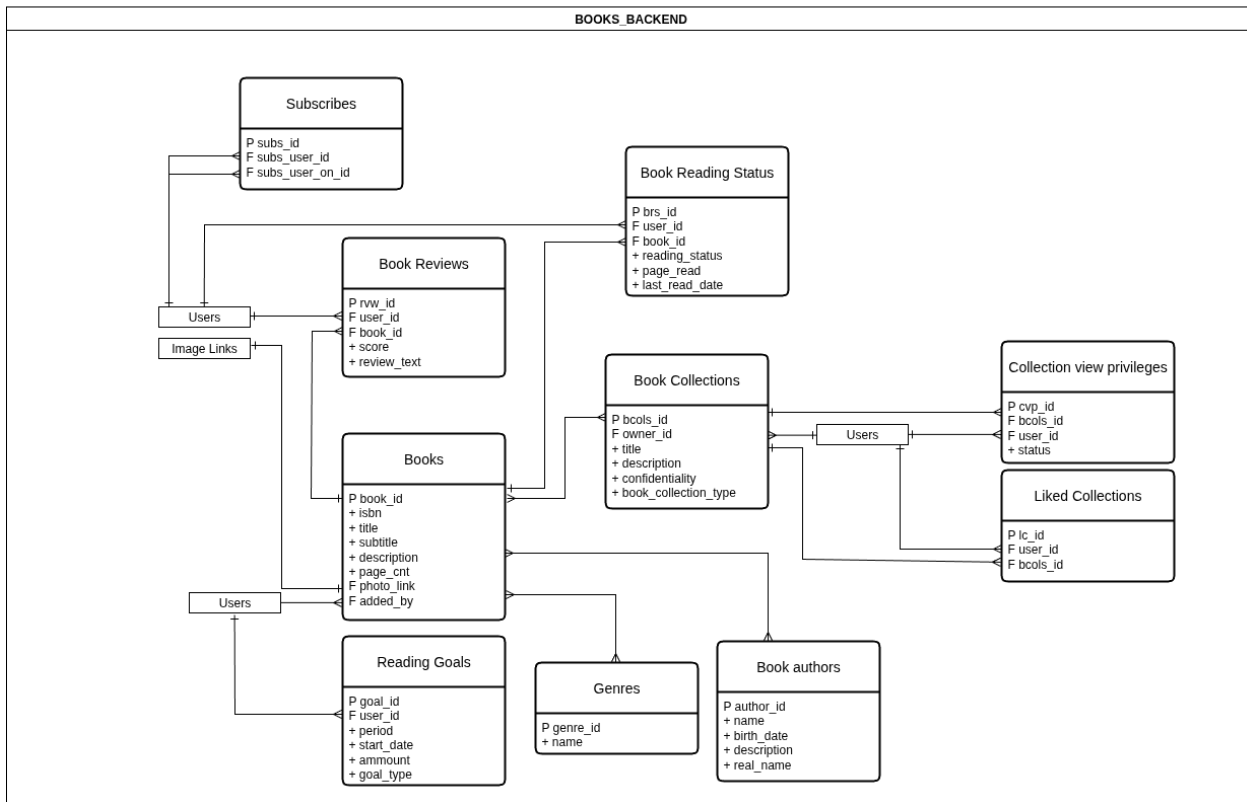
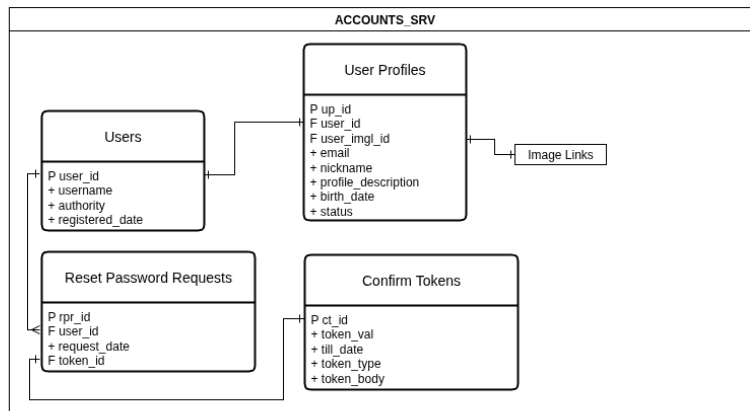
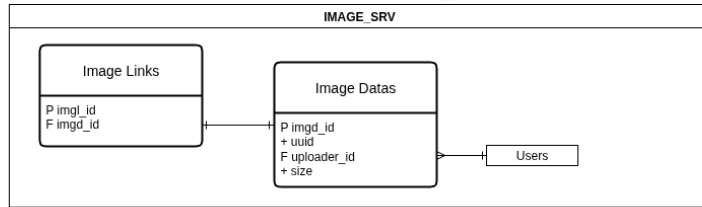
Выполнили: Брель Мария
Владимировна
Назин Артем Аркадьевич
Группа: Р3307
Преподаватель: Байрамова Хумай

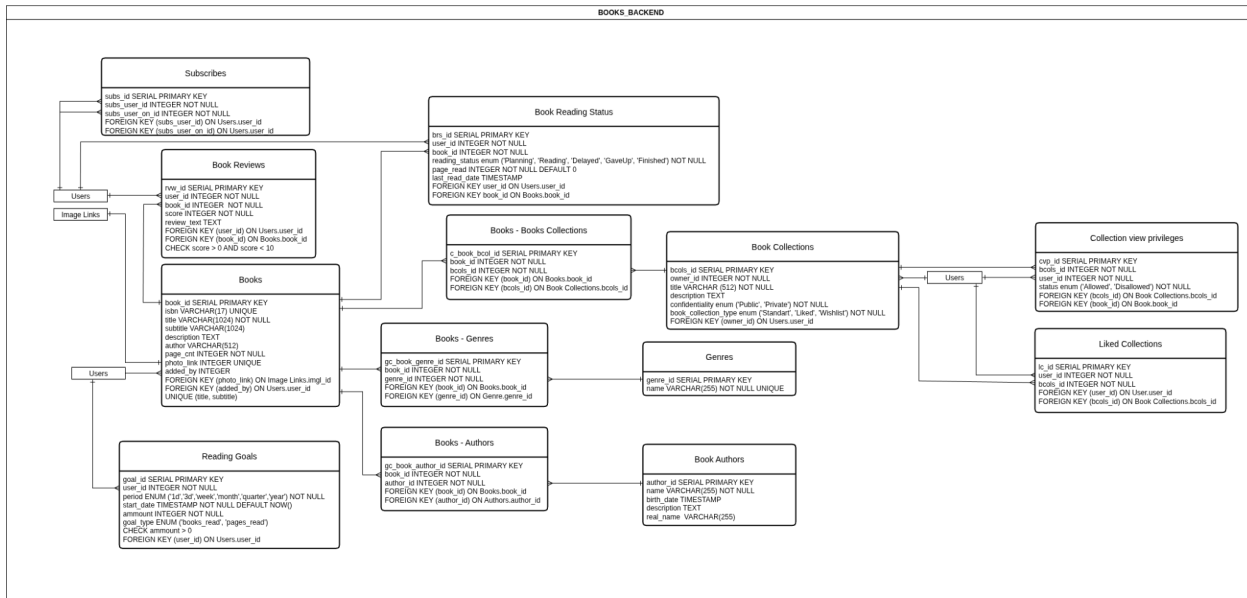
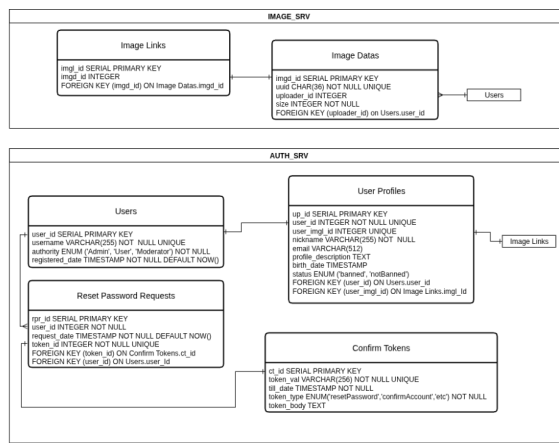
Город Санкт-Петербург
2025 год

Задание

1. Сформировать ER-модель базы данных (на основе описаний предметной области и прецедентов из предыдущего этапа).
ER-модель должна:
 - a. включать в себя не менее 10 сущностей;
 - b. содержать хотя бы одно отношение вида «многие-ко-многим».
2. Согласовать ER-модель с преподавателем. На основе ER-модели построить даталогическую модель.
3. Реализовать даталогическую модель в реляционной СУБД PostgreSQL.
4. Обеспечить целостность данных при помощи средств языка DDL и триггеров.
5. Реализовать скрипты для создания, удаления базы данных, заполнения базы тестовыми данными.
6. Предложить pl/pgsql-функции и процедуры, для выполнения критически важных запросов (которые потребуются при последующей реализации прецедентов).
7. Создать индексы на основе анализа использования базы данных в контексте описанных на первом этапе прецедентов. Обосновать полезность созданных индексов для реализации представленных на первом этапе бизнес-процессов.
8. Составить отчет.

Ход работы





Созданные индексы:

```
CREATE INDEX BOOK_INDEX ON BOOKS.BOOKS(title, subtitle);
CREATE INDEX USER_INDEX ON ACCOUNTS.USERS(username);
CREATE INDEX COLLECTION_INDEX ON BOOKS.BOOK_COLLECTIONS(title);
CREATE INDEX BOOK_AUTHORS_INDEX ON BOOKS.AUTHORS(name);
```

- 1) Индекс для поиска книги по её названию
- 2) Поиск пользователя по его никнейму
- 3) Поиск коллекции книг по её названию
- 4) Поиск автора книг по имени

Созданные функции/процедуры/триггеры:

```
CREATE OR REPLACE FUNCTION BOOKS.STATUS_UPDATE_FUNCTION()
RETURNS TRIGGER AS $$
DECLARE
    book_pages INTEGER;
BEGIN
    NEW.last_read_date = NOW();
    SELECT page_cnt INTO book_pages FROM BOOKS.BOOKS WHERE book_id = NEW.book_id;
    IF NEW.page_read = book_pages THEN
        NEW.reading_status = 'Finished';
    END IF;
END;
```

```

        END IF;
        RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER STATUS_UPDATE_TRIGGER
BEFORE UPDATE ON BOOKS.BOOK_READING_STATUS
FOR EACH ROW
EXECUTE FUNCTION BOOKS.STATUS_UPDATE_FUNCTION();

CREATE FUNCTION BOOKS.CAN_VIEW_COLLECTION(
    p_user_id INTEGER,
    p_collection_id INTEGER
) RETURNS BOOLEAN AS $$
DECLARE
    v_collection_confidentiality BOOKS.CONFIDENTIALITY;
    v_has_disallowed BOOLEAN;
    v_has_allowed BOOLEAN;
    v_owner INTEGER;
BEGIN
    SELECT confidentiality, owner_id
    INTO v_collection_confidentiality, v_owner
    FROM BOOKS.BOOK_COLLECTIONS
    WHERE bcols_id = p_collection_id;

    IF NOT FOUND THEN
        RETURN FALSE;
    END IF;

    IF p_user_id = v_owner THEN
        RETURN TRUE;
    END IF;

    SELECT
        EXISTS(SELECT 1 FROM BOOKS.COLLECTION_VIEW_PRIVILEGES
            WHERE bcols_id = p_collection_id
                AND user_id = p_user_id
                AND status = 'Disallowed'),
        EXISTS(SELECT 1 FROM BOOKS.COLLECTION_VIEW_PRIVILEGES
            WHERE bcols_id = p_collection_id
                AND user_id = p_user_id
                AND status = 'Allowed')
    INTO v_has_disallowed, v_has_allowed;

    IF v_collection_confidentiality = 'Public' THEN
        RETURN NOT v_has_disallowed;
    ELSIF v_collection_confidentiality = 'Private' THEN

```

```

        RETURN v_has_allowed;
    END IF;

    RETURN FALSE;
END;
$$ LANGUAGE plpgsql;

```

```

CREATE FUNCTION BOOKS.GET_VISIBLE_COLLECTIONS_FOR_USER(
    p_user_id INTEGER
) RETURNS TABLE(
    bcols_id INTEGER,
    owner_id INTEGER,
    title VARCHAR(512),
    description TEXT,
    confidentiality BOOKS.CONFIDENTIALITY,
    book_collection_type BOOKS.COLLECTION_TYPE
) AS $$
BEGIN
    RETURN QUERY
    SELECT
        bc.bcols_id,
        bc.owner_id,
        bc.title,
        bc.description,
        bc.confidentiality,
        bc.book_collection_type
    FROM BOOKS.BOOK_COLLECTIONS bc
    WHERE bc.owner_id = p_user_id
        AND BOOKS.CAN_VIEW_COLLECTION(p_user_id, bc.bcols_id) = TRUE

    UNION

    SELECT
        bc.bcols_id,
        bc.owner_id,
        bc.title,
        bc.description,
        bc.confidentiality,
        bc.book_collection_type
    FROM BOOKS.LIKED_COLLECTIONS lc
    JOIN BOOKS.BOOK_COLLECTIONS bc ON lc.bcols_id = bc.bcols_id
    WHERE lc.user_id = p_user_id
        AND BOOKS.CAN_VIEW_COLLECTION(p_user_id, bc.bcols_id) = TRUE;

END;
$$ LANGUAGE plpgsql;

```

Код

<https://github.com/ArtyomFuzis/ISKR/tree/master/iskr-database>

Вывод

В ходе выполнения второго этапа курсовой работы была сформирована модель базы данных, разработаны скрипты, написаны функции и процедуры, необходимые для последующей реализации прецедентов и созданы индексы.